King Abdulaziz University

Faculty of Computing and Information Technology

Department of Computer Science

CPCS-351 Group Project – First Semester 2026

Section: GAR

Project ID: Group 1

Student Name	Student ID
Leader: Arwa Ali AL-Qahtani	2306986
Rahaf Rajab AL-Zahrani	2307168
Ghadi Khalid Al-Ghamdi	2307159
Jood Faisal AL-Zahrani	2307623

Date	Version	Description	Authors
September 13th, 2025	0.1	Phase 1	Group 1
October 11th, 2025	0.2	Phase 2	Group 1
November 6th ,2025	0.3	Phase 3	Group 1
November 30rd ,2025	0.4	Phase 4	Group 1

Table of Contents

List Of Figures: 3

List Of Tables: 4

Phase 1: Project Description and Definition..... 5

1.1 Introduction.....5

1.2 Project Description and Objectives.....5

1.2.1 Project Description 5

1.2.2 Problem objectives..... 6

1.3 Project Team 7

1.4 Project goals.....7

1.5 Scope and Sources of Domain Analysis Information 7

1.5.1 Scope 7

1.6 Project Features.....8

1.6.1 Included Features 8

1.6.2 Excluded Features 9

Phase 2: Business Requirement Specification..... 10

2.1 System's Stakeholders, Actors, and Participants 10

2.2 Techniques for Gathering Data 10

2.3 Requirements & its types 11

2.3.1 Functional Requirements.....11

2.3.2 Nonfunctional Requirements..... 14

2.4 Use Case Diagram for Diyar.....15

2.5 Use Case Description.....15

2.5.1 First Use Case Description..... 15

2.5.2 Second Use Case Description..... 17

2.6 Sources of Domain Analysis Information.....19

Phase 3: Designing and Structuring 21

3.1 Domain Model 21

3.2 UML Class Diagram 21

3.3 Define Class Association and Multiplicity 22

3.3.1 Association Relationships 22

3.3.2 Aggregation Relationships 23

3.3.3 Composition Relationships 23

3.3.4 Generalization Relationships 23

Phase 4: Modeling, Interaction, and Behaviour Diagrams and Testing 24

4.1 Sequence Diagrams 24

4.1.1 First Sequence Diagram 24

4.1.2 Second Sequence Diagram 25

4.2 State Diagrams 27

4.2.1 First State Diagram: Login To The System 27

4.2.2 Second State Diagram: Make Rental Payments Online 28

4.3 Activity Diagram 29

4.3.1 First Activity Diagram: Search & Filter Properties 29

4.3.2 Second Activity Diagram: Submit Maintenance Request 30

4.4 System Architecture 31

4.4.1 Type of the System 31

4.4.2 System Design 31

4.4.3 System Components & hardware 32

4.5 Testing 34

4.5.1 Testing Objectives 34

4.5.2 Testing Strategy 34

4.5.2.1 Testing Levels 34

4.5.2.2 Testing Techniques 36

4.5.2.3 Pass / Fail Criteria 36

4.5.2.4 Excluded Features 37

4.5.2.5 Test Plan 1 37

4.5.2.6 Test Plan 2 38

4.5.3 Design Technique **40**

4.5.3.1 Equivalence Partitioning **40**

4.5.3.2 Boundary Value Analysis **40**

4.5.3.2 Decision Table Testing **40**

References 41

Appendix 42

List Of Figures:

FIGURE 1. USE CASE DIAGRAM 15

FIGURE2 DIYAR DOMAIN MODEL 21

FIGURE 3 DIYAR UML CLASS DIAGRAM 21

FIGURE 4. SEQUENCE DIAGRAM FOR THE "SEARCH AND FAVORITE PROPERTIES" USE CASE (UC-1) 24

FIGURE 5: SEQUENCE DIAGRAM FOR THE "SUBMIT MAINTENANCE REQUEST" USE CASE (UC-2) 25

FIGURE 6: STATE DIAGRAM FOR LOGIN TO THE SYSTEM. 27

FIGURE 7: STATE DIAGRAM FOR MAKING RENTAL PAYMENTS ONLINE. 28

FIGURE 8: DIAGRAM FOR THE "SEARCH & FILTER PROPERTIES" USE CASE (UC-1) 29

FIGURE 9: ACTIVITY DIAGRAM FOR THE "SUBMIT MAINTENANCE REQUEST" USE CASE (UC-2) 30

FIGURE 10. THE DISTRIBUTION OF PARTICIPANTS (LANDLORDS VS. TENANTS) 42

FIGURE 11. CHART SHOWING TENANT RESPONSES ON THE EASE OF CONTACTING LANDLORDS FOR HELP OR MAINTENANCE. 42

FIGURE 12. CHART SHOWING TENANT RESPONSES ON HOW QUICKLY THEY RECEIVE REPLIES AFTER REPORTING A PROBLEM. 43

FIGURE 13. SHOWING TENANT RESPONSES ON HOW OFTEN THEY RECEIVE UPDATES AFTER SUBMITTING MAINTENANCE REQUESTS. 43

FIGURE 14. CHART SHOWING TENANT RESPONSES ON RECEIVING IMPORTANT UPDATES FROM LANDLORDS. 43

FIGURE 15. CHART SHOWING TENANT SATISFACTION LEVELS WITH HOW MAINTENANCE REQUESTS ARE HANDLED. 44

FIGURE 16. CHART SHOWING TENANT INTEREST IN VIEWING A PROPERTY'S MAINTENANCE HISTORY BEFORE RENTING. 44

FIGURE 17. CHART SHOWS TENANT WILLINGNESS TO SHARE FEEDBACK OR RATE THEIR RENTAL EXPERIENCE AFTER LEAVING A PROPERTY. 44

FIGURE 18. CHART SHOWING LANDLORDS' PREFERRED METHODS OF COMMUNICATION WITH TENANTS. 45
FIGURE 19. CHART SHOWING LANDLORDS' RESPONSES ON THE EASE OF MANAGING COMMUNICATION WITH MULTIPLE TENANTS. 45
FIGURE 20. CHART SHOWING LANDLORDS' METHODS FOR TRACKING OR RECORDING MAINTENANCE REQUESTS. 45
FIGURE 21. CHART SHOWING LANDLORDS' RESPONSES ON THE DIFFICULTY OF HANDLING MULTIPLE MAINTENANCE REQUESTS SIMULTANEOUSLY. 46
FIGURE 22. CHART SHOWING LANDLORDS' RESPONSES ON KEEPING RECORDS OF TENANTS' PAYMENT OR BEHAVIOR HISTORY. 46
FIGURE 23. LANDLORDS' WRITTEN RESPONSES ON THE MOST TIME-CONSUMING TASKS IN MANAGING RENTAL PROPERTIES. 47

List Of Tables:

TABLE 1 FIRST USE CASE DESCRIPTION 17
TABLE 2 SECOND USE CASE DESCRIPTION 19
TABLE 3. TEST CASES OF TEST PLAN 1 38
TABLE 4. TEST CASES OF TEST PLAN 2 39
TABLE 5. EQUIVALENCE PARTITIONING TECHNIQUE 40
TABLE 6. BOUNDARY VALUE ANALYSIS TECHNIQUE 40
TABLE 7. DECISION TABLE TESTING TECHNIQUE 40

Phase 1: Project Description and Definition

1.1 Introduction

The rental housing market is the foundation of urban life, providing essential accommodation for thousands of people [1].However, the traditional rental process often involves several challenges for both landlords and tenants, including disorganized communication [2], delayed maintenance handling [3], unclear payment tracking, and limited transparency. Studies on property management reveal that miscommunication and maintenance delays are among the top causes of tenant dissatisfaction. Meanwhile, landlords often struggle with manual record-keeping and the inefficient monitoring of multiple properties simultaneously. [4]

As cities become bigger and the demand for rental units increases, it has become crucial to adopt smarter solutions to ensure efficiency and satisfaction for all parties involved. [5] Modern technology offers the potential to modify and change key rental operations, allowing for faster communication, better maintenance, secure payment tracking, and improved decision-making through real-time data [6].

To address these issues, our project introduces **Diyar (ديار)**, a Smart Rental Management System designed to make rental operations seamless, transparent, and stress-free. Diyar provides an integrated digital platform that connects landlords, tenants, and service providers, ensuring that all rental-related processes are efficient, compact, and easily accessible.

1.2 Project Description and Objectives

1.2.1 Project Description

Diyar is a smart rental management system that simplifies and makes the entire rental process smoother for both landlords and tenants. It enables landlords to manage multiple properties through a single platform by adding units with detailed specifications, photos, and locations. Tenants can search for properties that match their preferences using an AI-based recommendation system, resulting in a faster and more personalised experience.

The system provides clear and organised communication through built-in chat, allowing tenants to submit maintenance requests and complaints directly. Landlords can assign maintenance tasks, connect with service companies, and track the progress of requests, while both parties receive real-time notifications. An emergency mode allows tenants

to instantly alert landlords and maintenance providers in critical situations such as water leaks or power failures.

Diyar also includes tools to improve community interaction and responsibility. Landlords can send building-wide announcements, tenants can rate property and service quality, and landlords can rate tenant behaviour. Additional features include rent payment alerts, contract expiry reminders, and property history records give tenants and landlords a complete understanding of their responsibilities and rights. Interactive dashboards for both user types provide insights into statistics, complaints, satisfaction levels, and overall property performance, making rental management more efficient, transparent, and user-friendly.

1.2.2 Problem objectives

This project addresses several critical issues observed in current rental management practices. Through reviewing existing processes and tenant–landlord interactions, several recurring problems were identified that negatively affect communication, service quality, and response time. The main problem objectives can be summarized as follows:

1. **Tenant and Landlord Communication Is Ineffective:** Poor communication methods (such as phone conversations or emails), which are not trackable, are the root cause of many rental conflicts and delays.
2. **Lack of Transparency in Maintenance and Repairs:** Tenants often don't know when their maintenance requests will be handled, and landlords can't track progress effectively.
3. **Difficulty in Emergency Handling:** In urgent situations (like water leaks or power outages), there is no fast, unified system to notify both landlords and maintenance teams.
4. **Absence of a Centralized Notification and Reminder System:** Due to a lack of prompt reminders, tenants fail to pay rent on time or when their contracts expire; landlords find it difficult to manually manage several tenants.
5. **Uninformed Procedure for Choosing Properties:** Renters are unhappy and move frequently because of their inability to find properties that fit their demands and budget.

6. **Lack of Data-Driven Insights for Landlords:** Landlords don't have easy access to statistics on tenant satisfaction, property condition, or service quality, making it harder to improve operations.
7. **Fragmented Platforms for Rental Management:** Many landlords use multiple tools (e.g., spreadsheets, email, WhatsApp) instead of a unified platform, which leads to confusion and mismanagement.
8. **Inaccessible Maintenance Records for** tenants often leave them unaware of their property's past maintenance history, which can impact their decision to rent and lead to distrust or dissatisfaction later.

1.3 Project Team

During the first phase of the project, our team, Arwa Ali AL-Qahtani, Rahaf Rajab AL-Zahrani, Ghadi Khalid Al-Ghamdi, and Jood Faisal AL-Zahrani, collaborated on all tasks. We contributed to planning, research, and the initial design, applying each member's expertise to set objectives, organize tasks, and develop preliminary solutions for the project.

1.4 Project goals

Diyar Smart Rental Platform is designed to modernize and simplify the relationship between landlords and tenants, with a focus on transparency, efficiency, and user experience. It aims to provide a centralized solution that streamlines property management, improves communication, automates maintenance tracking, and facilitates emergency handling. By integrating features such as real-time chat, smart notifications, AI-based property recommendations, and rating systems, the platform reduces administrative burdens, supports informed decision-making, and enhances overall satisfaction for both landlords and tenants. This project seeks to deliver a reliable, intelligent, and user-friendly system that effectively addresses key challenges in the rental process [7].

1.5 Scope and Sources of Domain Analysis Information

1.5.1 Scope

Diyar will allow landlords to review their experience with tenants and benefit from previous reviews. In addition to managing property listings, tracking maintenance requests, assigning service tasks, and receiving emergency alerts. It will also help in

sending community notifications, reminders for rent payments, and contract expiry dates. Also, a dashboard displaying property statistics, maintenance status, and tenant ratings will be available to landlords.

For tenants, our platform will offer communication tools to contact landlords easily, submit maintenance requests, and receive emergency alerts. Tenants will also benefit from features like rent reminders, ratings and reviews of properties and services, and access to past maintenance records.

1.6 Project Features

1.6.1 Included Features

The system will provide the following features:

- **Property Management:** Include rental units with pictures, description, and address info so that Landlords can easily list their properties.
- **Communication & Requests:** Native chat, which allows landlords and tenants to communicate, including maintenance requests and complaints, directly.
- **Maintenance Scheduling:** Landlord's Solution – Assign service jobs to your team, associate requests with local businesses, and get notifications in real time on the state of them all.
- **Emergency Mode:** One-click emergency alerts about dire matters such as water leaks or power outages that alert both the landlord and the maintenance company immediately.
- **Community Notifications:** A Communication tool for landlords to broadcast announcements, reminders, or emergency notifications to every tenant in a building or unit.
- **Alerts & Reminders:** Automatic messages sent to tenants regarding upcoming rent payments and tenancy deadlines to ensure that no deadlines are missed.
- **Ratings & Reviews:** A shared review system where the tenants rate property/service quality and owners can review the tenants' behavior.
- **Interactive Dashboard:**
    1. **Landlord Dashboard:** It shows information about the property, complaints regarding tenants, and the happiness of all tenants.
    2. **Tenant Dashboard:** This would be a tool to monitor repair requests and other rental-related issues.

- **Filter-Based Search System:** The filtering system allows the tenant to easily find suitable properties by applying filters such as preferred location, budget range, number of rooms, and available services, ensuring that the search results match the tenant's selected criteria.
- **Property History:** Provides tenants with a full history of maintenance/repairs for their unit when moving in, fostering transparency and trust.

1.6.2 Excluded Features

The system will not provide the following features:

- **Legal Procedures:** The system does not replace legal processes like eviction and contention resolution, which continue to be the purview of official authorities.
- **Smart Home Integration:** Smart device (i.e., lighting, air conditioning) based functions are excluded.
- **Offline Functionality:** The app needs access to the Internet to be fully functional; offline capabilities are reduced.
- **Real House Market Place Forecasts:** The system does not forecast rent prices or the market conditions of supply and demand in real estate.

Phase 2: Business Requirement Specification

2.1 System's Stakeholders, Actors, and Participants

Our project includes the following primary stakeholders, all of whom play an essential role in ensuring the system's success and effectiveness.

- **Landlords:** They are the primary part of the system, listing, maintaining, and keeping an eye on their rental agreements and properties. authorizes requests from tenants, adds or removes properties, and keeps track of payments.
- **Tenants:** They look for properties, apply for rentals, pay, check the status of their rentals, sign digital contracts, and report emergencies using the system.
- **Maintenance Service Providers:** Contractors or technicians who reply to maintenance requests made via the system. They receive maintenance requests and update their status when the task is finished.

2.2 Techniques for Gathering Data

To build a clear and realistic understanding of the current rental process and user needs, we used different data-collection techniques. These techniques helped us gather information from multiple perspectives, verify our assumptions, and ensure that the system requirements are based on real practices rather than expectations only. The main techniques used are summarized below:

- **Review of Articles:** Relevant articles on rental processes and property management were consulted to enhance our understanding and ensure that the system aligns with real-world practices and legal standards, supporting compliance with regulatory requirements [8].
- **Observation:** Our team carefully observed how rental activities are currently managed in traditional settings. By monitoring the daily interactions between landlords, tenants, and administrators, several manual and repetitive processes were identified. These observations provided practical insights into which operations could be automated within the System to save time and reduce errors.
- **Surveys and Questionnaires:** In our project, we gathered data through a survey aimed at gathering users' measurable information, their needs, preferences, and the issues they commonly face during rental transactions [9]. The survey revealed that tenants face significant challenges in communication and maintenance management. Most tenants (38.3%) rarely receive updates from their landlords, while

only a small fraction (10%) always do, highlighting inconsistent communication. Satisfaction with maintenance request handling is low, with 73.3% of tenants dissatisfied or very dissatisfied, indicating the need for efficient tracking and resolution of maintenance issues. Furthermore, a large majority of tenants (86.7%) expressed the importance of checking a property's maintenance history before renting, emphasizing the demand for transparency. Reaching landlords for assistance is difficult for most tenants (70%), and half of the respondents report waiting more than three days for a response to reported problems. Updates on maintenance progress are infrequent, with 78.4% of tenants receiving little or no information. These findings suggest that our system should prioritize features such as real-time maintenance tracking, automated notifications, and streamlined communication tools to enhance tenant experience and satisfaction.

2.3 Requirements & its types

2.3.1 Functional Requirements

This section defines the main requirements of the proposed Diyar system. The requirements are derived from the identified problems and collected data, and they describe what the system must do to support tenants, landlords, and vendors. Functional requirements are listed first and are organized into main requirements (R1, R2, …) and their related sub-requirements (R1.1, R1.2, …).

R1. The system shall allow users to register an account.

- **R1.1** Users shall choose their type (tenants or landlords).
- **R1.2** The system shall ask tenants and landlords to enter their **full name**, **email address**, **phone number**, **ID,** and **password**.
- **R1.3** The system shall verify the phone number by sending a verification code via **SMS**.
- **R1.4** The system shall confirm the successful account creation and redirect the **Tenant** or **Landlord** to the appropriate dashboard.

R2. The system shall allow all registered tenants and landlords to log in to their accounts.

- **R2.1** The system shall allow tenants and landlords to log in using their **ID** and **password**.
- **R2.2** The system shall send a verification code via **SMS** to verify the tenant's or landlord's phone number.

- **R2.3** The system shall allow tenants and landlords to reset their passwords if they forget them, by sending a reset link to their registered email.

R3. The system shall allow tenants and landlords to manage their profile information.

- **R3.1** The system shall allow tenants and landlords to **update** their profile information (e.g., **phone number**, **email address**, and **password)**.
- **R3.2** The system shall allow tenants and landlords to **upload** and **update** their **profile picture**.
- **R3.3** The system shall allow tenants and landlords to **delete** their **profiles**.

R4. The system shall allow tenants to search for available properties based on selected filters.

- **R4.1** The system shall prompt tenants to enter their desired **property location**, **price range**, **number of bedrooms**, and **property type**.
- **R4.2** The system shall apply these filters to query the database for matching properties.
- **R4.3** The system shall allow tenants to filter results further based on **amenities** (e.g., parking, gym, pool).
- **R4.4** The system shall rank and display the filtered results based on **property ratings**, from the highest to the lowest rating, allowing tenants to view the most highly rated properties first.

R5. The system shall allow tenants to save and favorite properties.

- **R5.1** The system shall allow tenants to **save** properties by clicking the "Save" button.
- **R5.2** The system shall allow tenants to view and manage their **favorite properties**.

R6. The system shall allow tenants to contact property landlords directly.

- **R6.1** The system shall display **landlord contact information** (phone number or email) once a property is selected.
- **R6.2** The system shall allow tenants to send **direct messages** to landlords within the system.
- **R6.3** The system shall notify landlords when a tenant sends them a message.

R7. The system shall allow tenants to submit maintenance requests for their rented properties.

- **R7.1** The system shall prompt tenants to describe the **maintenance issue** and select its **priority** (Low, Medium, High).
- **R7.2** The system shall prompt tenants to **upload images** or **videos** (at least two total) to document the issue.
- **R7.3** The system shall notify **landlords** of the maintenance request.
- **R7.4** The system shall allow the landlord to change the status of the maintenance request (e.g., Pending, In Progress, Completed), and the updated status shall be displayed to the tenant in real-time.
- **R7.5** The system shall allow tenants and landlords to send and control **emergency alerts** and ensure that **immediate action** is taken to resolve the issue.

R8. The system shall allow tenants to make rental payments online.

- **R8.1** The system shall display the **payment due date** for each active lease.
- **R8.2** The system shall allow tenants to **pay rent online** using integrated payment methods (e.g., credit card, bank transfer).
- **R8.3** The system shall send **payment reminders** to tenants 3 days before the due date.

R9. The system shall allow tenants and landlords to rate and review each other.

- **R9.1** The system shall allow tenants to **rate properties** on a scale of 1 to 5 stars after the rental period ends.
- **R9.2** The system shall allow tenants to leave **written reviews** regarding the property and landlord.
- **R9.3** The system shall allow **landlords** to rate tenants after the rental period ends.

R10. The system shall allow tenants and landlords to log out of their accounts.

- **R10.1** The system shall provide a **"Logout"** button on the **dashboard**.
- **R10.2** The system shall log the **tenant** or **landlord** out and redirect them to the **login screen** once the "Logout" button is clicked.

R11. The system shall allow landlords to manage their property listings.

- **R11.1** The system shall allow landlords to **add** new properties to their listing.
- **R11.2** The system shall allow landlords to define the rental type (e.g., monthly or daily), set the rent price, and enter detailed property information, including Number of bedrooms, Property location (neighborhood, city, area), Type of rental agreement (short-term or long-term), and Property amenities (e.g., parking, gym, pool).
- **R11.3** The system shall allow landlords to **edit** existing property details (e.g., rent price, amenities).
- **R11.4** The system shall allow landlords to **delete** a property from their listing.

2.3.2 Nonfunctional Requirements

Nonfunctional requirements describe the quality attributes and constraints of the Diyar system rather than specific features. They ensure that the system is usable, reliable, secure, and accessible for all users in different environments. These are the main non-functional requirements:

- **R1** The system shall run smoothly on all major browsers and devices.
- **R2** The system shall support both Arabic and English languages.
- **R3** The system shall provide a simple, intuitive, and user-friendly interface.
- **R4** The system shall provide both light and dark modes.
- **R5** Users shall be able to complete the main actions within three clicks.
- **R6** The system shall encrypt all sensitive data using strong encryption methods.
- **R7** All user data and actions shall be automatically saved and recoverable after any failure.
- **R8** The system should support at least 100,000 concurrent users without performance issues.
- **R9** The system shall be accessible at all times.
- **R10** The system shall use a modular design to make maintenance, updates, and adding new features easy.

2.4 Use Case Diagram for Diyar

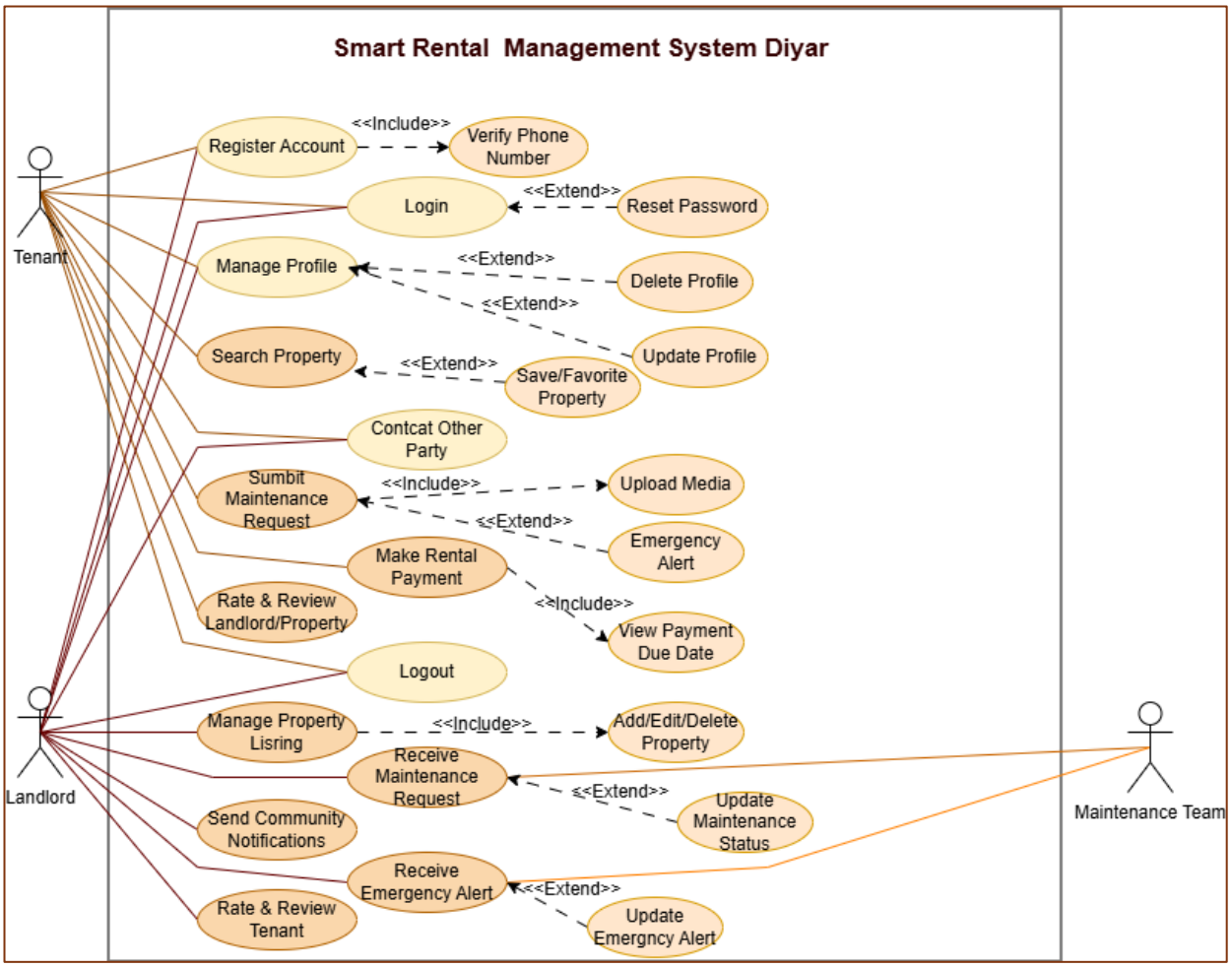

Figure 1. Use Case Diagram

2.5 Use Case Description

This section presents the detailed use case descriptions for the Diyar system. Each use case specifies how different actors interact with the system to achieve a particular goal, including the main flow of events, alternative scenarios, and possible exceptions.

2.5.1 First Use Case Description

Use Case Name:	**Search and Filter Properties**
Use Case ID:	UC-1
Description:	This use case describes how a tenant can search for available rental properties in the ***Diyar*** system using filtering options such as location, price, number of rooms, and property type. The system enables tenants to

	narrow down their search results by applying multiple filters, allowing them to find the most suitable property that matches their specific needs.
Primary Actor:	Tenant
Supporting Actors:	None
Preconditions:	• The tenant must be registered and logged into the system. • There must be at least one available property listed in the system database.
Postconditions:	• The system displays all properties that match the tenant's search criteria. • The tenant can view detailed property information, save listings to favorites, or contact the landlord directly.
Main Scenario:	1. Tenant opens the "Search" page from the main menu. 2. The system displays available filters (location, price range, property type, number of rooms, and amenities). 3. Tenant selects preferred filters such as city, maximum rent, and number of bedrooms. 4. The tenant clicks the "Apply Filters" button. 5. The system retrieves and displays properties that meet all the selected criteria. 6. Each result shows key information such as property image, price, address, and a brief description. 7. Tenant can click on a property to view full details (landlord info, amenities, and maintenance record). 8. Tenant can save the property to the "Favorites" list or contact the landlord directly. 9. The system keeps a record of the search filters for easier future access.
Extensions:	**4a. No Filters Selected** 4a.1. The Tenant clicks "Apply Filters" without selecting any criteria. 4a.2. The system detects that no filters were applied.

	4a.3. The system retrieves and displays all available properties (Continues to Step 5). **5a. No Matching Properties Found** 5a.1. The system queries the database for matching properties but returns an empty set. 5a.2. The system displays the message: "No properties found. Try changing the filters." 5a.3. The Tenant is returned to the filter selection page. **8a. Save Property to Favorites** 8a.1. In Step 8, the Tenant clicks the "Save" button. 8a.2. The system adds the selected property to the Tenant's "Favorites" list. **8b. Contact Landlord** 8b.1. In Step 8, the Tenant clicks the "Contact Landlord" button. 8b.2. The system opens a chat or contact page to facilitate direct communication (R6.2).

Table 1 First Use Case Description

2.5.2 Second Use Case Description

Use Case Name:	**Submit Maintenance Request**
Use Case ID:	UC-2
Description:	This use case describes how a logged-in (registered) tenant can officially submit a maintenance request or complaint about their rented property to the landlord through the system. The system manages the request details, emergencies, attachments (pictures and videos), and real-time status updates.
Primary Actor:	Tenant
Supporting Actors:	Landlord, Maintenance Service Provider
Preconditions:	• The tenant must be registered and logged into the system. • The Tenant must have an active lease linked with a property in the system.

	• The system must be connected to the Maintenance Service Provider network (for emergency alerts).
Postconditions:	• The maintenance request is logged into the system. • The Landlord is notified of the request. • The Tenant is able to view the request status in their dashboard.
Main Scenario:	1. The Tenant navigates to the maintenance/requests section of their dashboard. 2. The Tenant selects the option to "Submit a New Maintenance Request." 3. The system prompts the Tenant to describe the issue. 4. The Tenant enters a description of the maintenance issue. 5. The system prompts the Tenant to select the priority: Low, Medium, or High. 6. The Tenant selects the appropriate priority level. 7. The system prompts the Tenant to upload images or videos of the issue. 8. The Tenant uploads at least two images/videos of the problem (as required by R7.2). 9. The Tenant clicks the "Submit Request" button. 10. The system validates all required fields and records the request as "Pending." 11. The system notifies the Landlord of the new maintenance request. 12. The Tenant is redirected to their request list where the new request status is displayed.
Extensions:	**9a. Critical Emergency Request (Emergency Mode)** 9a.1 The Tenant encounters a critical emergency (e.g., water leak or power failure). 9a.2. The Tenant clicks the "Emergency Mode" one-click alert. 9a.3. The system immediately sends an instant alert to the Landlord and the Maintenance Service Provider simultaneously. 9a.4. The system logs the alert and ensures immediate action is taken to resolve the issue.

	10a. Invalid Input 10a.1. In Step 9, the Tenant clicks "Submit Request," but one or more required fields (e.g., description, priority) are missing. 10a.2. The system displays an error message highlighting the missing fields. 10a.3. The use case returns to Step 3. **11a. Low-Priority Request (Landlord Action)** 11a.1. After Step 11, the Landlord reviews the "Pending" request. 11a.2. The Landlord assigns the request status to "In Progress." 11a.3. The system immediately updates the Tenant's view with the "In Progress" status. 11a.4. The Landlord marks the request as "Completed." 11.5. The system updates the Tenant's view with the "Completed" status and prompts the Tenant to review the service (see R9.2).

Table 2 Second Use Case Description

2.6 Sources of Domain Analysis Information

To better understand the rental domain and refine the requirements of the Diyar system, several sources of information were used. These sources helped us study existing solutions, stakeholder needs, and real-world practices, ensuring that our system design is both relevant and practical. The main sources used in the domain analysis are outlined below:

1. **Similar Existing Systems:** To gain a clear understanding of the market, we analyzed several existing rental and property management applications (e.g., **Booking**, **Gathern**, and **Airbnb)** [10] [11] [12]**.** By examining these platforms, we identified the gaps and unmet needs that both tenants and landlords continue to face, including limited direct communication, a lack of transparent maintenance tracking, and the absence of behavioral records.
2. **End Users (Landlords and Tenants):** To better understand the real needs and challenges faced by end users, we conducted a study involving both landlords and tenants. Their responses helped us identify the most common issues, including slow maintenance handling, poor communication, and a lack of reminders or status updates.

These insights guided the design of our proposed system to fill those gaps and enhance the overall rental experience for both parties.

Phase 3: Designing and Structuring

Link for better view for all diagrams:

3.1 Domain Model

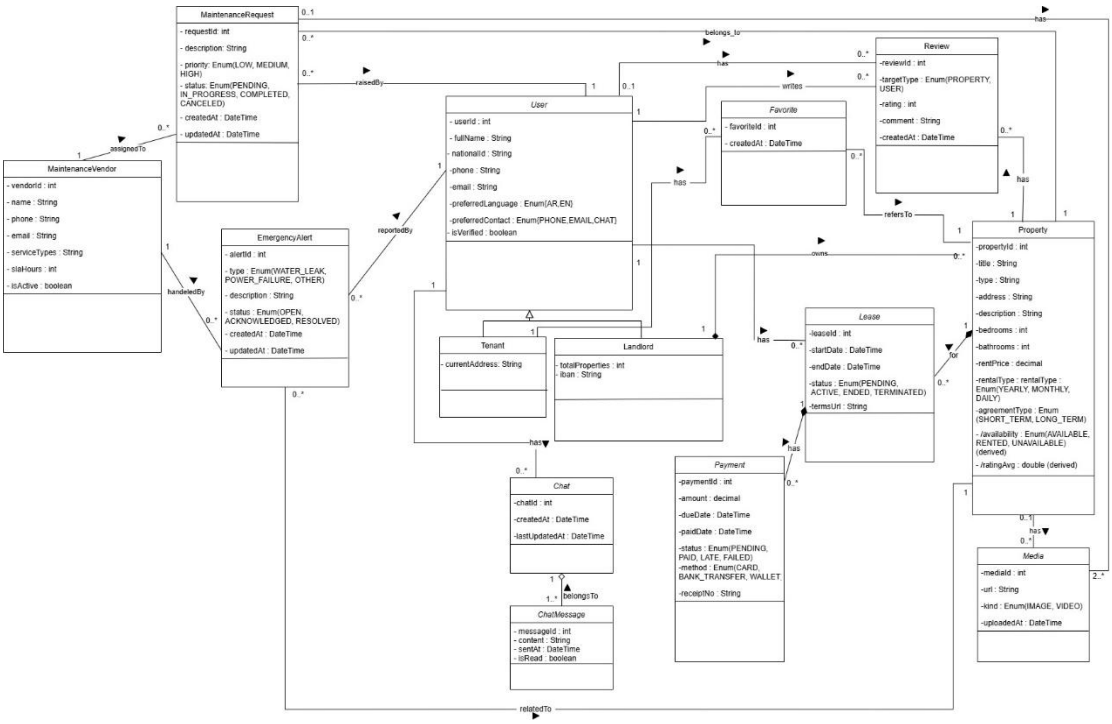

Figure2 DIYAR Domain Model

3.2 UML Class Diagram

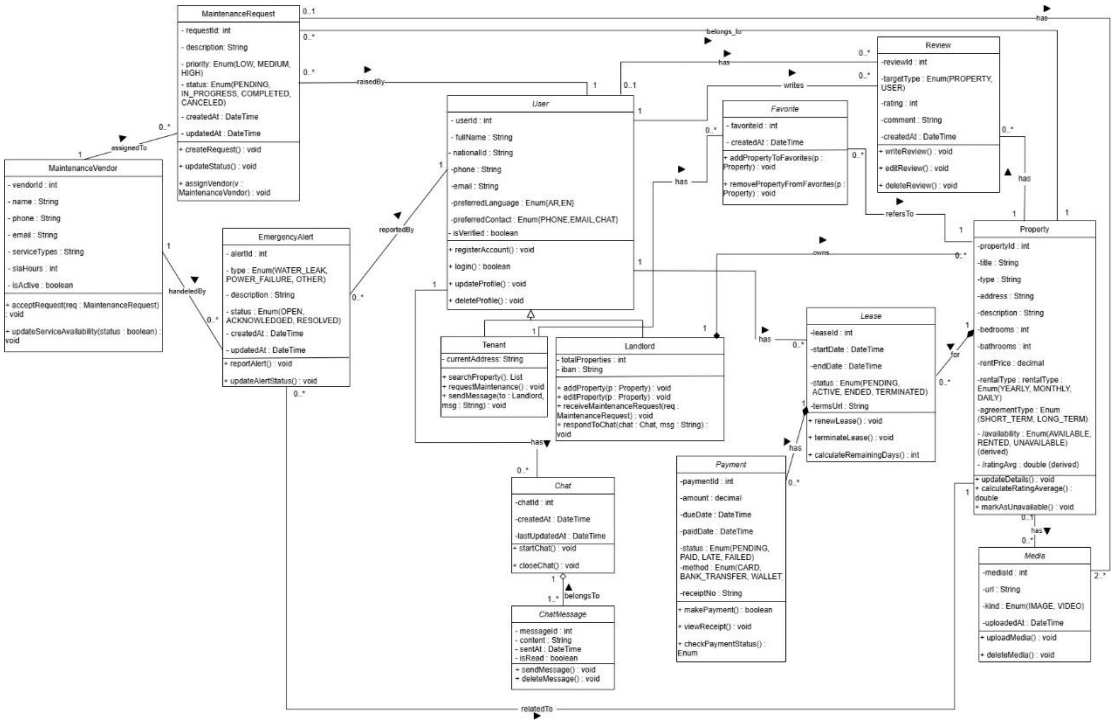

Figure 3 DIYAR UML Class Diagram

3.3 Define Class Association and Multiplicity

3.3.1 Association Relationships

1. Between Tenant class and Favorite class
    - One Tenant has 0 or more Favorite Property.
    - Each Favorite Property belongs to 1 Tenant.

2. Between User class and Review class
    - One user can write 0 or more Reviews.
    - Each Review is written by 1 User.

3. Between User class and Review class
    - One user can have 0 or more Reviews.
    - Each Review is owned by 1 User.

4. Between User class and EmergencyAlert class
    - One User can report **0 or more** Alerts.
    - Each EmergencyAlert is reported by **1 User.**

5. Between User class and MaintenanceRequest class
    - One User can raise **0 or more** MaintenanceRequests.
    - Each MaintenanceRequest is raised by **1 User**.

6. Between User class and Chat class
    - one User can participate in 0 or more chat.
    - Each Chat belongs to exactly 1 User.

7. Between User class and Lease class
    - One User can have 0 or more leases.
    - Each Lease belongs to 1 user.

8. Between Favorite class and Property class
    - one Favorite Refers to 1 Property.
    - Each property can appear in zero or more Favorites.

9. Between Media class and Property class
    - One Property can have 0 or more Media files
    - Each Media can belong to exactly 1 Property.

10. Between Review class and Property class
    - One Property can have zero or more Reviews.
    - Each Review refers to one Property.

11. Between EmergencyAlert class and Property class
    - One Property can have zero or more Emergency Alerts.
    - Each Emergency Alert is linked to one Property.

12. Between MaintenanceRequest class and Property class
    - One Property can have zero or more Maintenance Requests.
    - Each Maintenance Request is related to one Property.

13. Between MaintenanceRequest class and Media class
    - One Maintenance Request can have at least 2 Media files.
    - Each Media file is attached to one Maintenance Request.

14. Between MaintenanceRequest class and MaintenanceVendor class
    - One MaintenanceVendor can handle zero or more Maintenance Requests.
    - Each Maintenance Request can be assigned to one MaintenanceVendor.

15. Between EmergencyAlert class and MaintenanceVendor class
    - One MaintenanceVendor can handle zero or more Emergency Alerts.
    - Each Emergency Alert can be handled by zero or one MaintenanceVendor.

3.3.2 Aggregation Relationships

1. Between ChatMessage class and Chat class
    - One Chat can contain one or more ChatMessages.
    - Each ChatMessage belongs to exactly one Chat.

3.3.3 Composition Relationships

1. Between Lease class and Property class
    - One Property can have zero or more Leases.
    - Each Lease is associated with one Property.

2. Between Lease class and Payment class
    - One Lease can have zero or more Payments.
    - Each Payment belongs to one Lease.

3. Between Landlord class and Property class
    - One Landlord can own zero or more Properties.
    - Each Property is owned by one Landlord.

3.3.4 Generalization Relationships

1. Between User class and Landlord class
    - The Landlord class inherited User class.

2. Between User class and Tenant class
    - The Tenant class inherited User class.

Phase 4: Modeling, Interaction, and Behaviour Diagrams and Testing

4.1 Sequence Diagrams

4.1.1 First Sequence Diagram

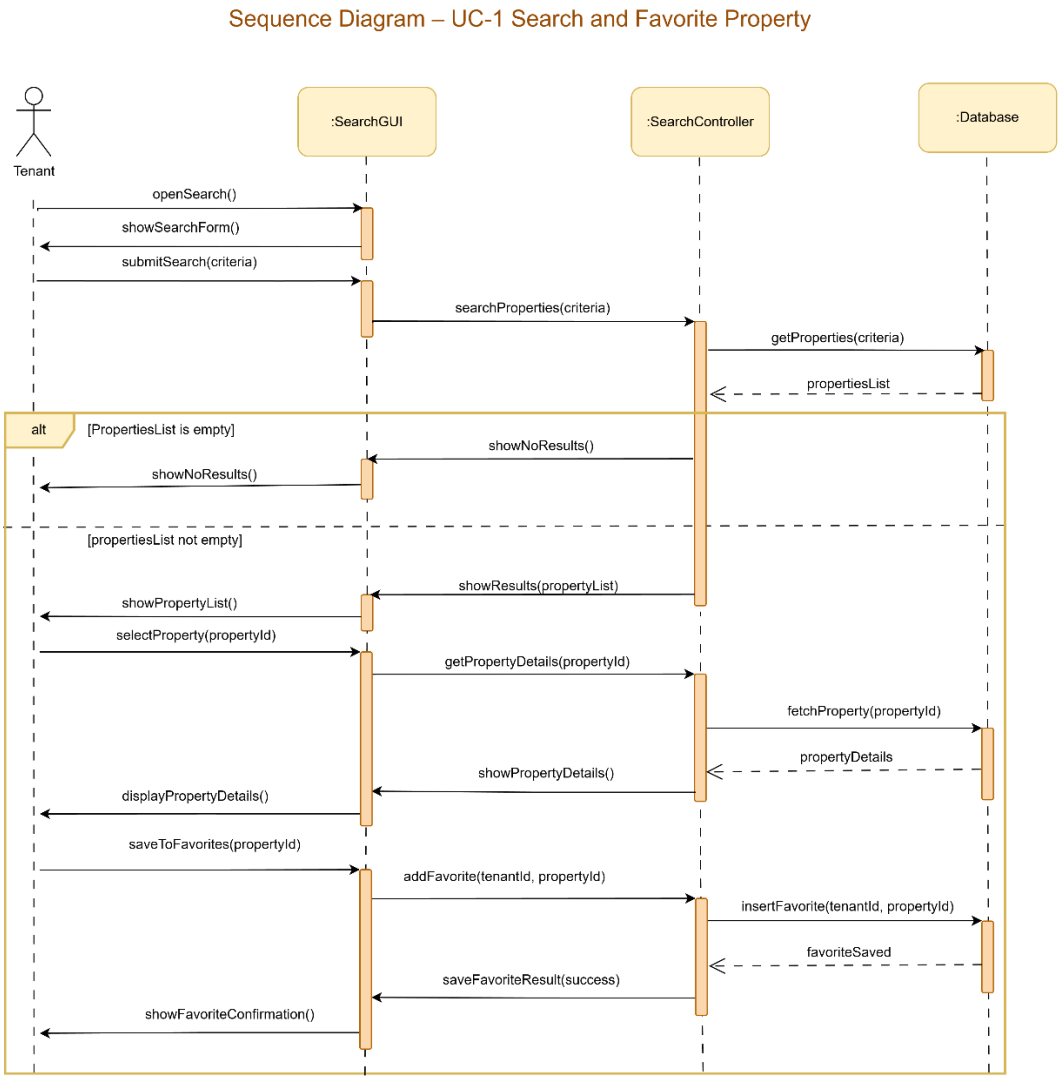

Figure 4. Sequence diagram for the "Search and favorite properties" use case (UC-1).

The sequence diagram shows how a tenant searches for properties and saves one as a favorite. The tenant submits search criteria, and the controller retrieves matching properties from the database. An **alt** fragment handles two outcomes: if no results are found, the system displays a message; otherwise, the property list is shown. The tenant can then view property details and save the selected property to favorites, after which a confirmation message is displayed.

4.1.2 Second Sequence Diagram

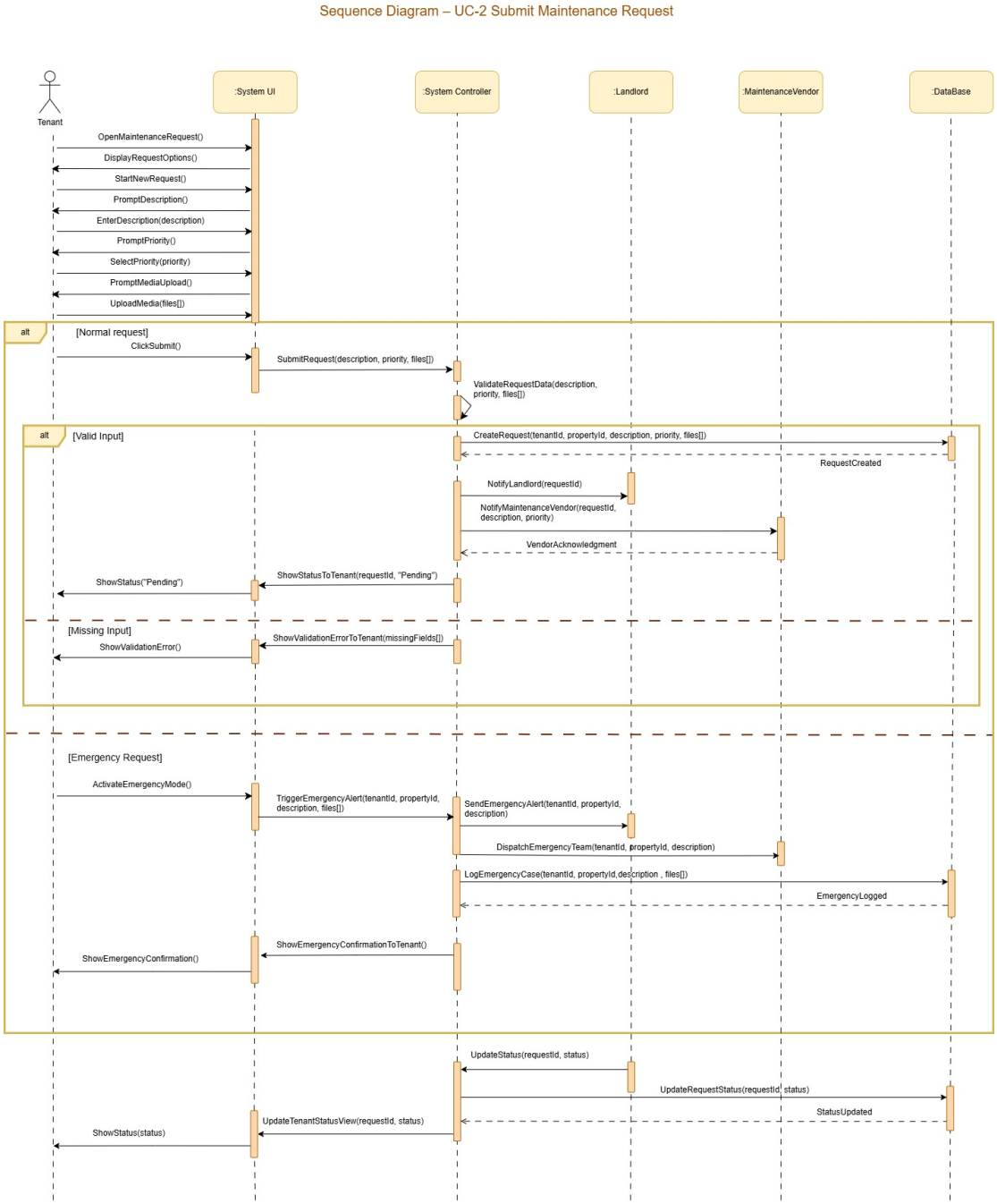

Figure 5: Sequence diagram for the "Submit Maintenance Request" use case (UC-2).

The sequence diagram illustrates the complete flow of the "Submit Maintenance Request (UC-2)" process, showing how a tenant submits a maintenance request through the system interface, for both regular and emergency cases. The diagram highlights the main components involved: the user interface, system controller, landlord, maintenance provider, and database, as well as the flow of data and message exchanges between them.

It includes steps for validating the input data, creating a maintenance request record in the database, sending notifications to the landlord and maintenance provider, and providing feedback to the tenant. In emergency cases, the diagram also depicts additional actions, such as triggering emergency alerts and dispatching the emergency team.

The diagram further covers the request status update process by the landlord, where updates are handled by the system controller, recorded in the database, and reflected to the tenant via the user interface. Overall, the diagram provides a clear and organized view of the sequence of operations involved in submitting, tracking, and managing maintenance requests within the system.

4.2 State Diagrams

4.2.1 First State Diagram: Login To The System

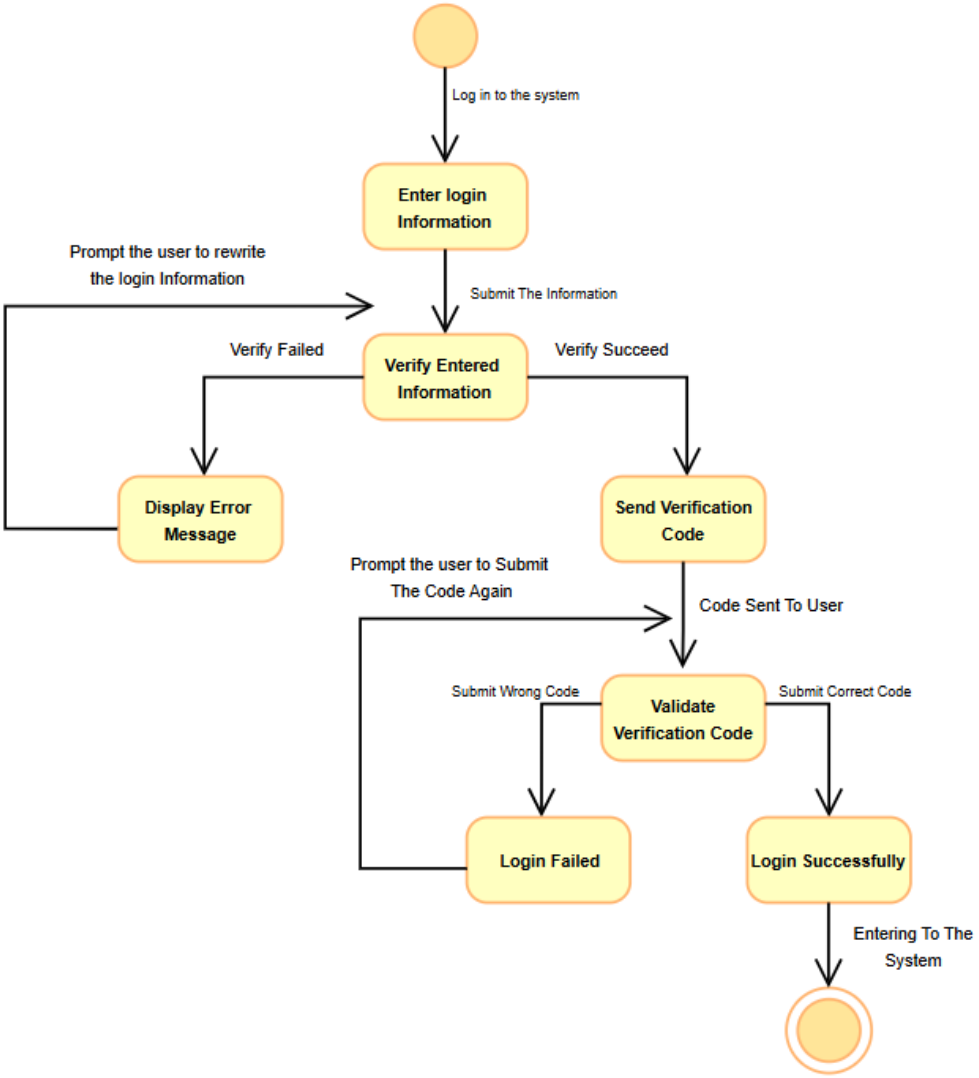

Figure 6: State Diagram For Login To The System.

This state diagram outlines the steps a user follows to securely access Diyar Smart Rental Management System. The process begins when the user enters their login information, then moves to the Verify Entered Information state. If the credentials are incorrect, the system displays an error message and prompts the user to re-enter their details. When the information is correct, the system transitions to Send Verification Code, sending a code to the user for additional security. The user then enters the code in the Validate Verification Code state. If the code is wrong, the system leads to Login Failed and asks the user to try again. If the code is correct, the flow proceeds to Login Successfully, granting the user access to the system.

4.2.2 Second State Diagram: Make Rental Payments Online.

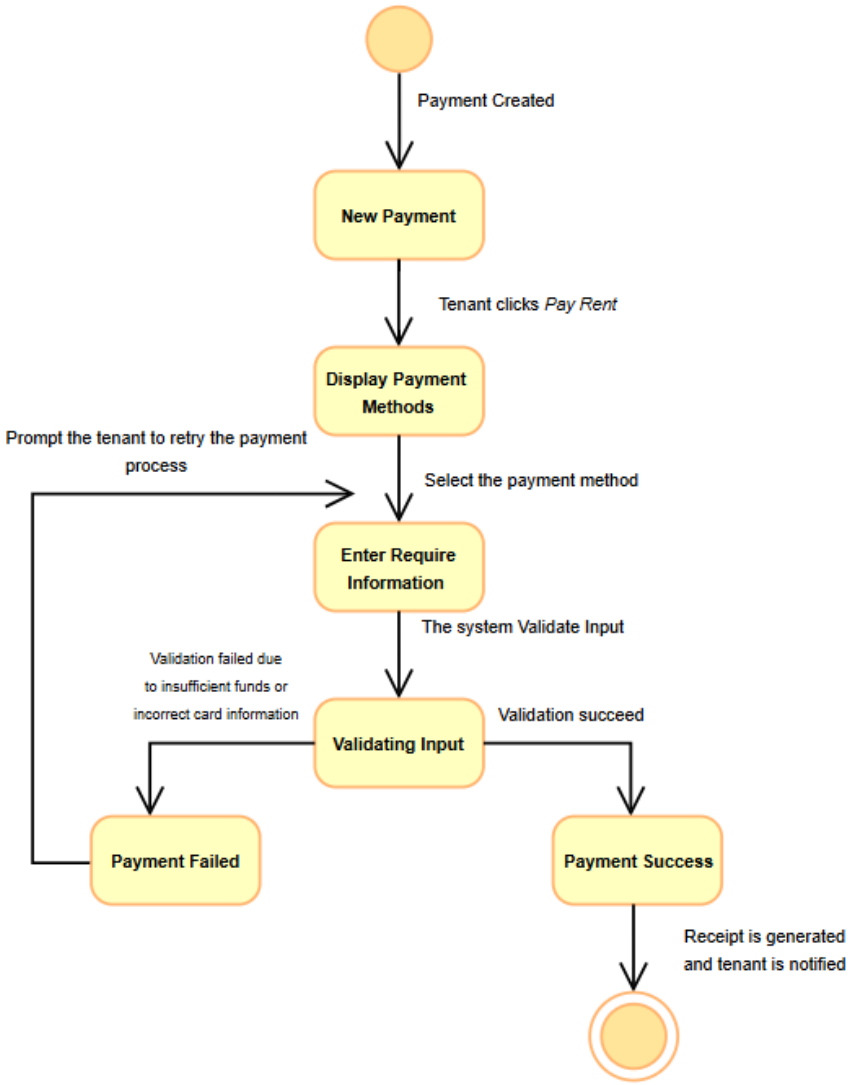

Figure 7: State diagram for making rental payments online.

This state diagram shows how a tenant completes a rent payment in Diyar Smart Rental Management System. The process starts with a New Payment, when the system generates a new payment linked to the current activated lease. Once the tenant selects Pay Rent, the system proceeds to Display Payment Methods, allowing the tenant to choose a preferred method. After the tenant selects a method and enters the required information, the system moves to Validating Input. If the information is incorrect or the payment fails, the system transitions to Payment Failed and prompts the tenant to retry. If validation succeeds, it proceeds to Payment Success, where a receipt is generated, and the tenant is notified.

4.3 Activity Diagram

The following activity diagrams illustrate the dynamic workflows of two core functionalities in Diyar Smart Rental Management System. Each diagram visually represents the sequence of actions, decision points, and interactions between the system and its users, based on the detailed use of case specifications outlined earlier in this report. The first activity diagram demonstrates how tenants search for rental properties using filters such as location, price, and amenities, corresponding to Use Case UC-1 and Functional Requirement R4. The second activity diagram models the process of submitting a maintenance request, including the emergency mode extension, as described in Use Case UC-2 and supported by Functional Requirement R7. Together, these diagrams provide a clear understanding of the system's operational behavior from the user's perspective

4.3.1 First Activity Diagram: Search & Filter Properties

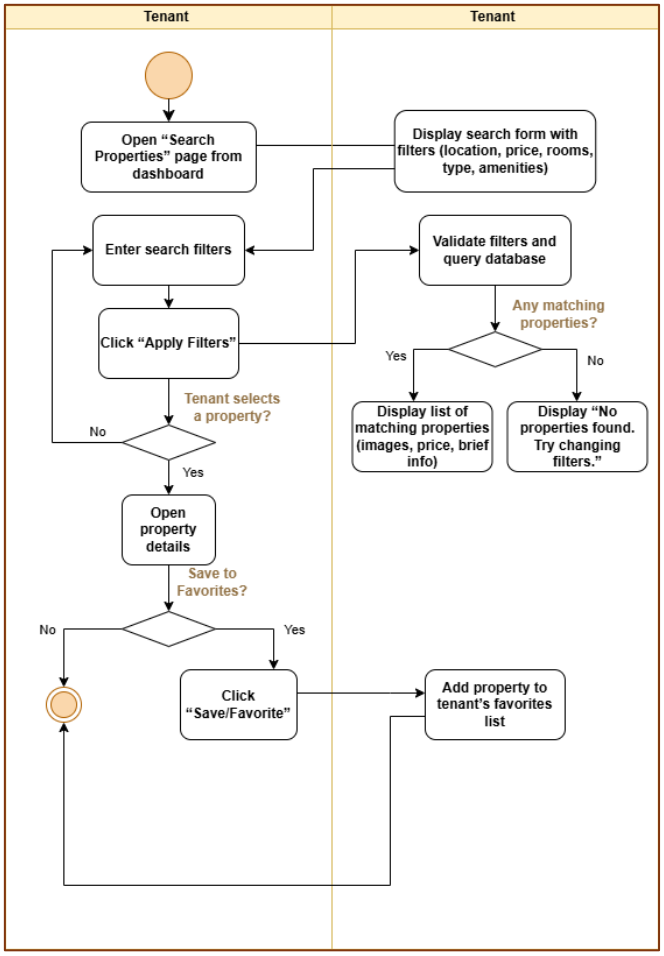

Figure 8: diagram for the "Search & Filter Properties" use case (UC-1).

This diagram shows how a tenant searches for rental properties by entering filters such as location, price, number of bedrooms, and amenities. It includes the system's response to matching and non-matching results and optional actions like saving the property. This corresponds directly to Use Case UC-1 and satisfies Functional Requirement R4.

4.3.2 Second Activity Diagram: Submit Maintenance Request

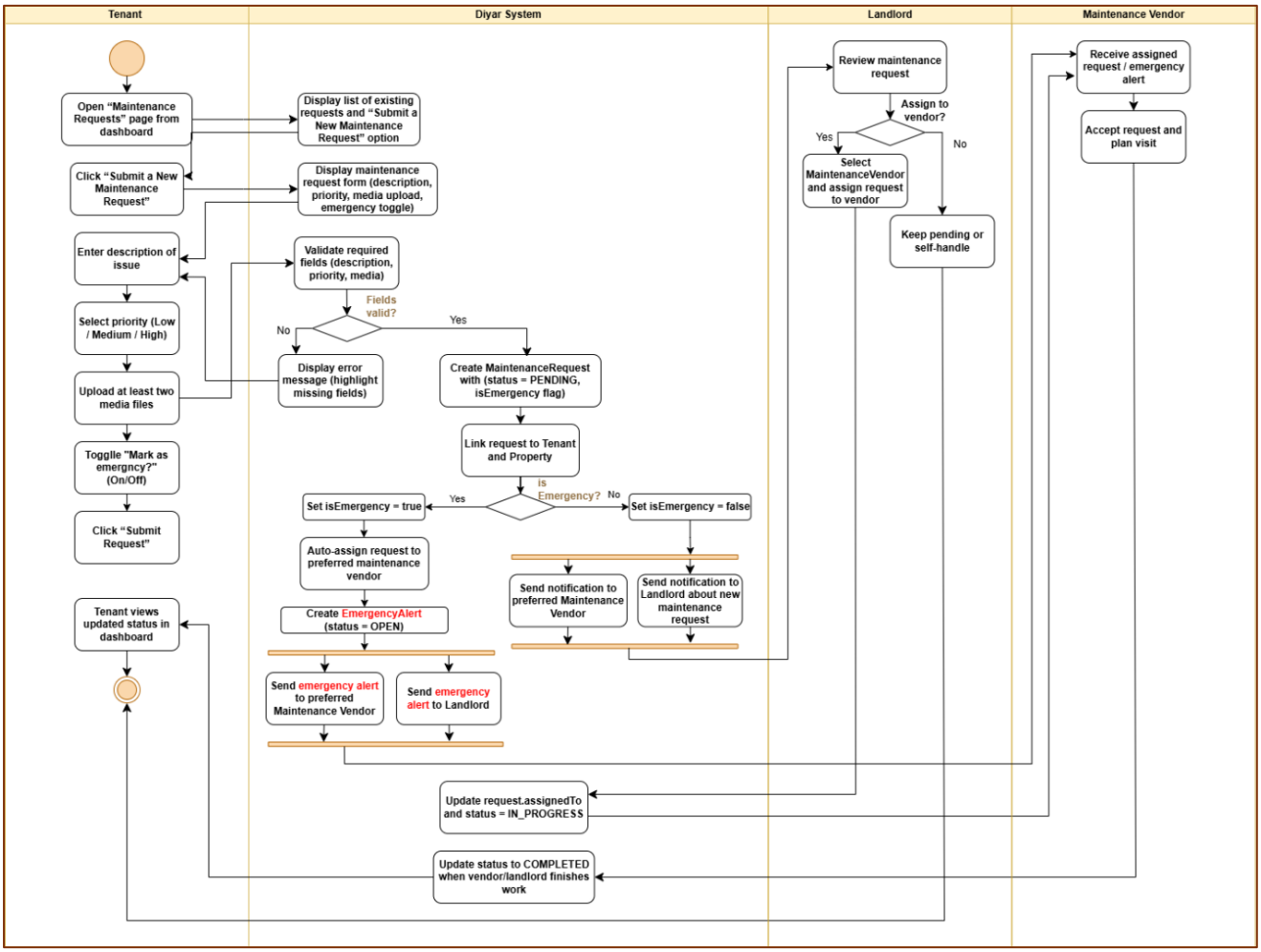

Figure 9: Activity diagram for the "Submit Maintenance Request" use case (UC-2).

This diagram illustrates the flow of actions involved when a tenant submits a maintenance request, including entering issue details, selecting priority, uploading media, and triggering Emergency Mode. It reflects the main scenario and extension conditions described in Use Case UC-2 and fulfills Functional Requirement R7.

4.4 System Architecture

4.4.1 Type of the System

DIYAR is an interactive rental property management system that acts as a unified platform connecting landlords, tenants, and maintenance vendors. The system aims to transform the traditional and complex rental management process into a transparent, efficient, and secure digital experience.

From property listings and digital contract signing to maintenance requests, notifications, and communication to electronic rent collection and full record storage for all parties, the system offers extensive features for managing the whole rental lifecycle.

4.4.2 System Design

1. Determine Design Objective

The design objective of the Diyar Smart Rental Management System is to provide a robust and secure platform for managing the rental process. Which ensures high scalability to effectively handle a large and growing number of concurrent users and transactions without performance degradation. The system is also designed for reliability and continuity, especially for time-critical features like emergency mode.

2. Determine Type of The System

Diyar is an interactive system.

3. Apply an Architecture Design

Diyar follows an N-tier style

4. Specify Subsystem Function, Interfaces and Interaction Behavior

Subsystems represent modules that support Landlords, Tenants, and Maintenance Vendors. The functional requirement describes every interface and function of the subsystems, while the UML diagram explains every interaction.

5. Review The Architectural Design

All specifications, architectural design goals, and constraints are satisfied. Software design principles are met by Diyar Architecture Design.

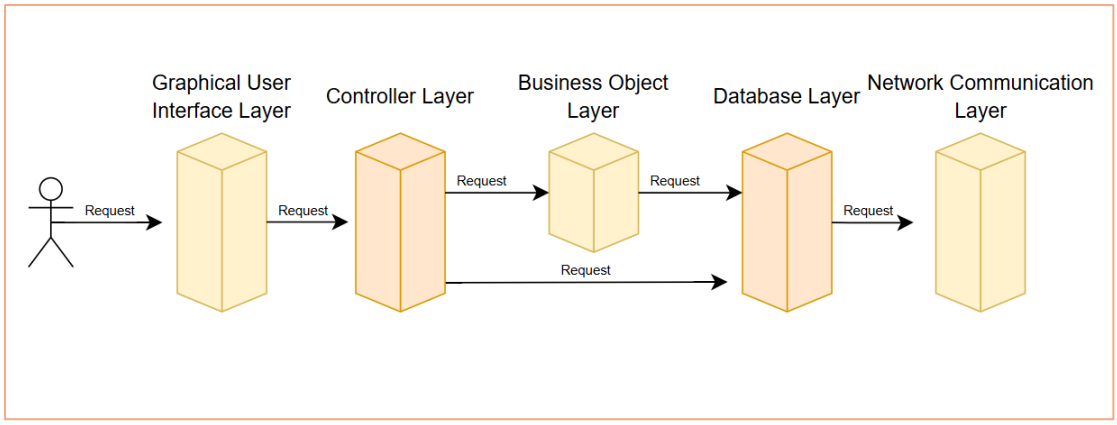

Figure 10: Architectural Design

4.4.3 System Components & hardware

Component:

1. Graphical User Interface Layer

   provides the main point of contact for all users, including maintenance vendors, landlords, and tenants. Users can see properties, sign digital leases, submit maintenance requests, process payments, and receive notifications using its user-friendly and responsive online and mobile displays. Usability, accessibility, and device consistency are key design considerations.

2. Controller Layer

   functions as the intermediary between the interface and the core system operations. When a user submits a request, the controller validates the input, manages session handling, and determines the appropriate action required. It ensures that all incoming data is properly checked and routed to the relevant business components while maintaining system-wide coordination and workflow control.

3. Business Object Layer

contains all logic and rules that define how the rental system operates. This layer models essential entities such as properties, payments, leases, users, emergency alerts, and maintenance requests. It applies the system's validation rules, performs calculations, updates statuses, and enforces business policies to guarantee consistency and correctness across all operations.

4. Database Layer
   is responsible for storing all long-term system data, including user accounts, property details, payment transactions, leases, chat messages, maintenance records, and emergency alerts. It ensures efficient data retrieval, supports concurrent users, maintains data integrity, and performs indexing and optimization to enhance performance.

5. Network Communication Layer

manages secure communication between the client devices and the backend services. Using encrypted protocols such as HTTPS, this layer is responsible for delivering requests reliably and protecting data during transmission. It enables smooth communication with external services such as payment gateways, notification services, or vendor systems when required.

Hardware:

The hardware required to operate the Diyar Smart Rental Management System consists primarily of an application server, a database server, client devices, and supporting network infrastructure. The application server hosts the backend logic and handles incoming user requests, requiring sufficient memory to maintain system performance. The database server stores all persistent information, such as user data, payments, leases, and maintenance records; it should use high-performance storage, and reliable backup mechanisms to ensure fast access and data protection. Users access the system through standard client devices like smartphones, which only require internet connectivity and a web browser. Finally, essential network hardware—including routers, switches, and firewalls support secure, stable communication and ensure encrypted data transmission between all components.

4.5 Testing

4.5.1 Testing Objectives

The main objective of the testing process is to ensure that the **Diyar System** delivers a fully functional, secure, and defect-free experience for all system users including Tenants, Landlords, and Maintenance Vendors. Testing confirms that the system meets its defined requirements and performs as expected under different conditions.

The testing objectives include the following:

- **Verifying that the system meets all intended functional and non-functional requirements**, such as submitting maintenance requests, updating request status, managing emergencies, and ensuring proper communication between users.

- **Detecting defects and errors early**, ensuring they are properly documented, fixed, and re-tested before deployment.

- **Ensuring the overall quality, reliability, and performance of the system**, including responsiveness, stability, and accurate real-time notifications.

- **Validating system behavior** across different usage scenarios, including normal maintenance requests and high-priority emergency requests, to ensure the system behaves correctly in all cases.

- **Improving usability and the interaction experience** of all actors (Tenant, Landlord, and Maintenance Vendor) through seamless system navigation and an intuitive UI design.

These objectives help ensure that the **Diyar System** is ready for deployment and can effectively support real users in managing maintenance operations with high reliability and user satisfaction.

4.5.2 Testing Strategy

4.5.2.1 Testing Levels

A. Unit Testing

Unit testing focuses on verifying individual components in isolation before they interact with other modules. It ensures that the smallest parts of the system behave as expected.

In Diyar, unit testing is applied to:

- Search filter logic
- Input validation functions (registration, login, maintenance requests)
- Property rating calculations
- Saving property to favorites
- Emergency alert triggering
- Small utility functions inside controllers

Unit testing helps detect errors early in development and ensures the internal logic of each module works properly.

B. Integration Testing

Integration testing validates that modules interact correctly when combined. Since Diyar relies on communication between UI, controllers, and the database, integration testing ensures that data flows smoothly across all layers.

Integration testing is applied to:

- Login and authentication flow
- Search and filter → Controller → Database
- Submit maintenance request → Media upload → Storage
- Favorite property → Controller → Database
- Review and rating submission
- Notifications and alerts

This step ensures that previously tested units, when connected, continue to function correctly.

C. System Testing

System testing evaluates Diyar as a complete product and ensures that all requirements are met from an end-user perspective. It validates the behavior of the entire system under realistic scenarios.

System testing includes:

- Searching and filtering properties
- Viewing property details
- Saving properties to favorites
- Submitting maintenance requests
- Emergency mode activation
- Adding reviews and ratings
- Logging in and out

The goal is to verify that all system features work together coherently.

4.5.2.2 Testing Techniques

A. Black-Box Testing

Black-box testing focuses on inputs and outputs without considering internal code structure. It is ideal for testing user-facing features and is used widely in Diyar's integration and system testing.

Black-box testing is applied to:

- Login inputs (valid/invalid email, password)
- Search filters (different values, empty values, combined filters)
- Boundary values (price range, number of rooms, rating 1–5)
- Maintenance request submission (missing fields, different priority levels)
- Viewing and saving favorites

Common black-box techniques used include:

- **Equivalence Partitioning** (e.g., valid vs. invalid search filter input ranges)
- **Boundary Value Analysis** (e.g., min/max price values, rating limits)
- **Decision Table Testing** (e.g., login input combinations)

B. White-Box Testing

White-box testing focuses on internal code logic, conditions, loops, and execution paths. It is used primarily during unit testing.

White-box testing is applied to:

- Internal filter logic (condition coverage)
- Repeated loops that build the filtered property list
- Error-handling paths when invalid input is submitted
- Validation functions for forms
- Controller methods that manipulate property or maintenance data

This ensures all internal branches, conditions, and paths are tested.

4.5.2.3 Pass / Fail Criteria

A test is considered **PASS** when:

- The actual output matches the expected output
- The correct screen, message, or action occurs
- No crashes, freezes, or unexpected behavior appears
- Data is stored and retrieved correctly
- The process follows the intended use case steps
- The system responds within reasonable time for normal operations

A test is considered **FAIL** when:

- The output does not match expectations
- An error message appears incorrectly
- The system crashes or hangs
- Invalid input is accepted
- Data is saved incorrectly or lost
- The screen or action does not follow the required workflow

These criteria help determine whether each functional and non-functional requirement has been met.

4.5.2.4 Excluded Features

The following features are excluded from the current testing scope: smart home device integration, legal and governmental procedures, offline system functionality, real payment gateway processing, SMS-based verification, large-scale performance or stress testing, and real-estate prediction or forecasting features

4.5.2.5 Test Plan 1

System: Diyar – Smart Rental Management System

Test Case Name: Search and Favorite Property

Description: Test whether the system correctly filters property results based on user-entered criteria and allows saving a property to favorites.

Test Scenario: Validate correct search behavior for matching and non-matching filters, and confirm proper functionality for saving a property to favorites.

Precondition:

- Tenant is registered and logged in.
- System contains properties with known attributes.

Test Cases:

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Results	Actual Results	Pass/ Fail
1	Search with matching properties and save to favorites	1. Log in as tenant. 2. Go to **Search Properties**. 3. Enter filters that match existing properties. 4. Click **Search**.	Valid filters that match a known property.	Matching properties are displayed. Selected property opens correctly. Property is added to favorites, and a	As expected	Pass

		5. Open a property. 6. Click **Save to Favorites**.		confirmation message appears.		
2	Search with no matching properties	1. Log in as tenant. 2. Go to **Search Properties**. 3. Enter filters that match no properties. 4. Click **Search**.	Filters are designed to return zero results.	System shows a clear "No properties found" message and displays no list. Saving a property is not possible.	As expected	Pass
3	System returns incorrect search results	1. Log in as tenant. 2. Go to **Search Properties**. 3. Enter specific filters (e.g., price 1000–1500, 2 bedrooms). 4. Click **Search**. 5. Observe displayed properties.	Specific filters that should exclude certain properties.	Only properties matching **all** filters should appear. No unmatched property should be displayed.	Some displayed properties do **not** match the filters.	Fail

Table 3. Test Cases of Test Plan 1

Postcondition: Search results are displayed based on the applied filters. Favorites are saved only when valid matching properties exist.

4.5.2.6 Test Plan 2

System: Diyar – Smart Rental Management System

Test Case Name: Submit Maintenance Request

Description: Test whether tenants can successfully submit maintenance requests and whether the system properly handles invalid or failing submissions.

Test Scenario: Validate correct request creation when required data is provided and proper error handling for invalid or failed submissions.

Precondition:

- Tenant is logged in.
- Tenant has at least one active property/lease in the system.

Test Cases:

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Results	Actual Results	Pass/Fail
1	Submit a valid maintenance request	1. Log in as tenant. 2. Navigate to **My Properties** and select a property. 3. Click **Submit Maintenance Request**. 4. Fill all required fields. 5. Click **Submit**.	Valid issue type, description, priority, 2 images.	Request is created successfully with status "Pending" and appears in the tenant's request list. A success message is displayed.	As expected	Pass
2	Attempt to submit with missing required fields	1. Log in. 2. Select property. 3. Open **Submit Maintenance Request**. 4. Leave one or more required fields empty, less than 2 images attached 5. Click **Submit**.	Missing required fields (e.g., empty description or issue type).	System displays validation error messages. No maintenance request is created.	As expected	Pass
3	System fails to create request despite valid input	1. Log in. 2. Select property. 3. Open **Submit Maintenance Request**. 4. Enter all required fields correctly. 5. Click **Submit** while a backend/database failure is simulated.	All valid fields entered correctly	System should create the request and show a success message.	Request not created; system error or no response.	Fail

Table 4. Test Cases of Test Plan 2

Postcondition: For valid inputs, requests are created and stored. For invalid or failing submissions, no request is created, and appropriate feedback is displayed.

4.5.3 Design Technique

4.5.3.1 Equivalence Partitioning

Scenario: Rating Value

Invalid	Valid	Invalid
… -2 rating -1 rating 0 rating	1 rating 2 rating … 5 rating	6 rating 7 rating 8 rating …

Table 5. Equivalence Partitioning Technique

4.5.3.2 Boundary Value Analysis

Scenario: Password length requirement

Invalid Partition – Valid Partition Lower Boundary		Invalid Partition – Valid Partition Upper Boundary	
Boundary value just below the boundary	Boundary value just above the boundary	Boundary value just below the boundary	Boundary value just above the boundary
7 Characters	8 Characters	32 Characters	33 Characters

Table 6. Boundary Value Analysis Technique

4.5.3.2 Decision Table Testing

Scenario: Update Profile (Email & Phone Number)

Email format valid?	**T**	**T**	**F**	**F**
Phone number valid?	**T**	**F**	**T**	**F**
Expected Result	**Update successfully**	**Reject; "Invalid phone number."**	**Reject; "Invalid email format."**	**Reject; "Invalid email and phone number.**

Table 7. Decision Table Testing Technique

References

[1] Y. Q. Y. L. L. H. Y. F. S. Y. J. &. Z. D. Li, "Effect of increasing the rental housing supply on house prices: Evidence from China's large and medium-sized cities," *Land Use Policy,* vol. 123, p. 106420, 2022.

[2] C. P. M. (. PM), ""Pitfalls of Old School Landlords"," Classic Property Management (Classic PM). [Online]. [Accessed 2025].

[3] A. L. S. P. Corporation, ""What Are the Biggest Challenges Faced by Landlords and Tenants?"," Ahmed Legal Services Professional Corporation, [Online]. Available: https://ahmedlegalservices.ca/what-are-the-biggest-challenges-faced-by-landlords-and-tenants/#:~:text=Late%20or%20Missed%20Rent%20Payments,Tenant%20Board%20to%20be%20resolved.. [Accessed 11 October 2025].

[4] S. Garg, " "5 Common Challenges and Solutions for Landlords"," Crib, [Online]. Available: https://www.cribapp.com/resources/landlord-challenges-solutions.

[5] N. Conisbee, ""The Complex Challenge of Taming Rental Growth"," [Online]. Available: https://www.linkedin.com/pulse/complex-challenge-taming-rental-growth-nerida-conisbee-46ljc.

[6] A. Help, ""How Technology Is Transforming the Property Management Industry"," APM Help, [Online]. Available: https://www.apmhelp.com/blog/how-technology-is-transforming-the-property-management-industry.

[7] "first Defining Project Goals and Objectives," [Online]. Available: https://www.projectsmart.co.uk/smart-goals/defining-project-goals-and-objectives.php?utm_source=chatgpt.com#google_vignette.

[8] "second Systematic Literature Review to Identify the Critical Success Factors of the Build-to-Rent Housing Model," [Online]. Available: https://www.mdpi.com/2075-5309/12/2/171?utm_source=chatgpt.com.

[9] "Diyar Survey," [Online]. Available: https://forms.gle/QaJ3ZPJT2Bijpukq6.

Appendix

Our survey questions and responses are as follows:

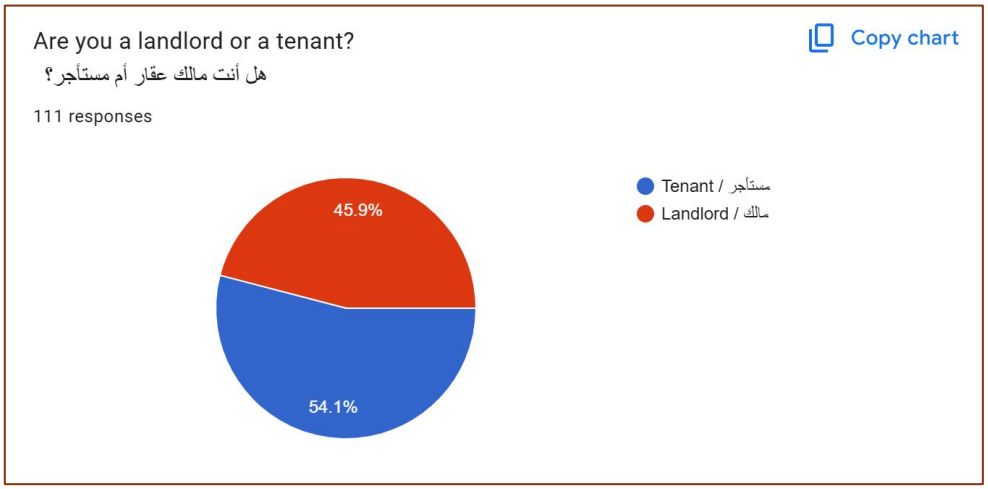

Figure 10. The distribution of participants (landlords vs. tenants)

Tenants' Answers

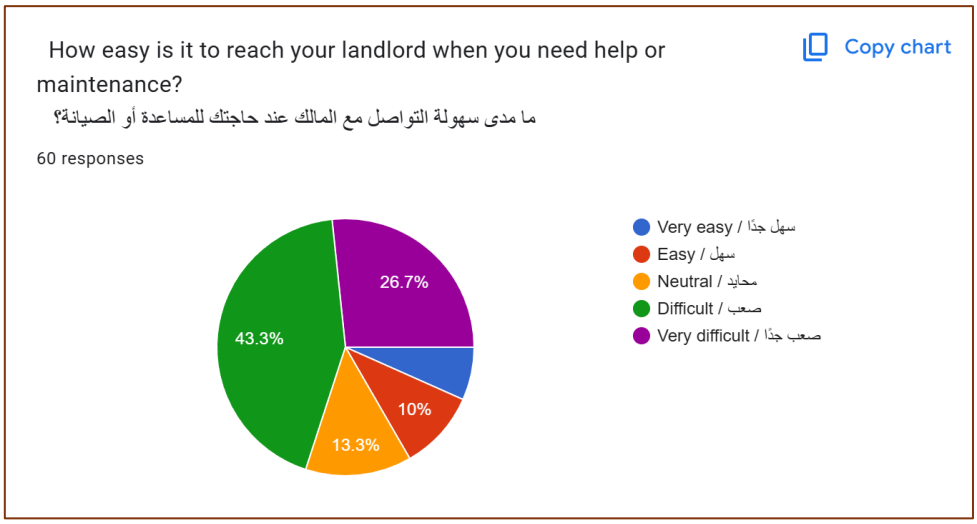

Figure 11. Chart showing tenant responses on the ease of contacting landlords for help or maintenance.

Figure 12. Chart showing tenant responses on how quickly they receive replies after reporting a problem.

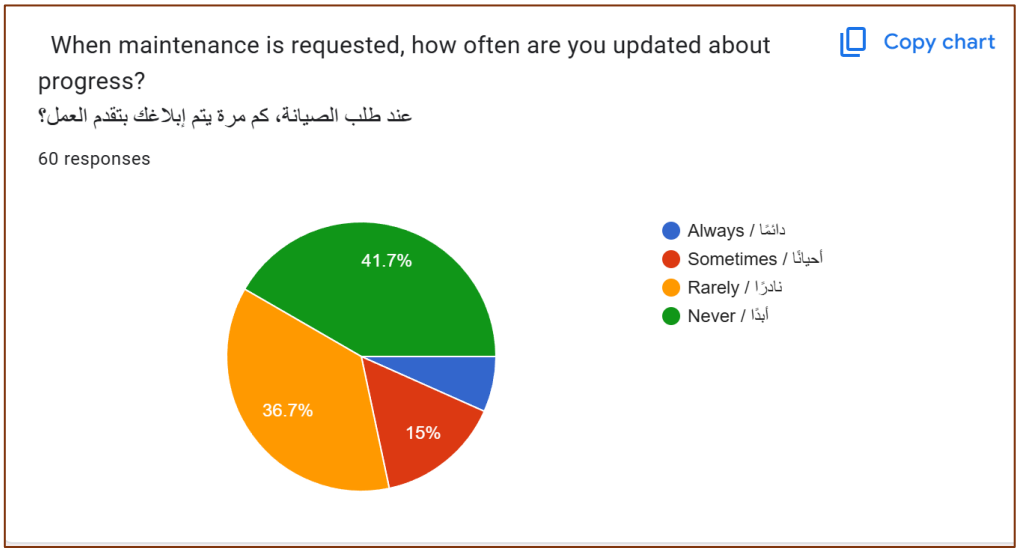

Figure 13. showing tenant responses on how often they receive updates after submitting maintenance requests.

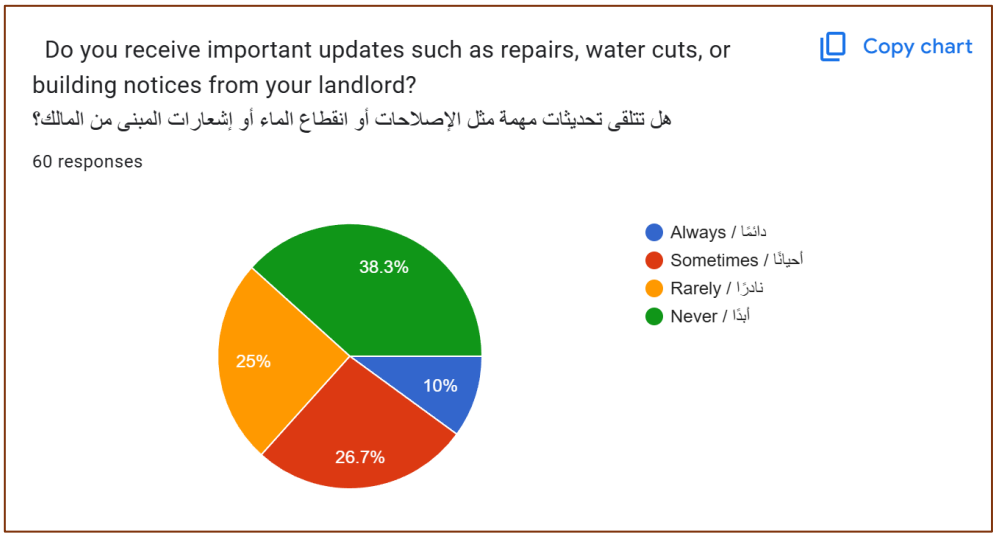

Figure 14. Chart showing tenant responses on receiving important updates from landlords.

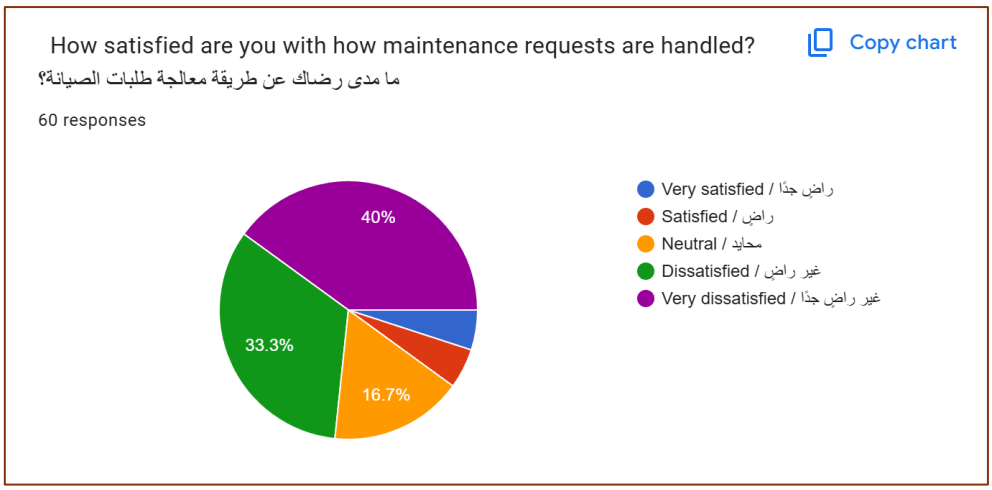

Figure 15. Chart showing tenant satisfaction levels with how maintenance requests are handled.

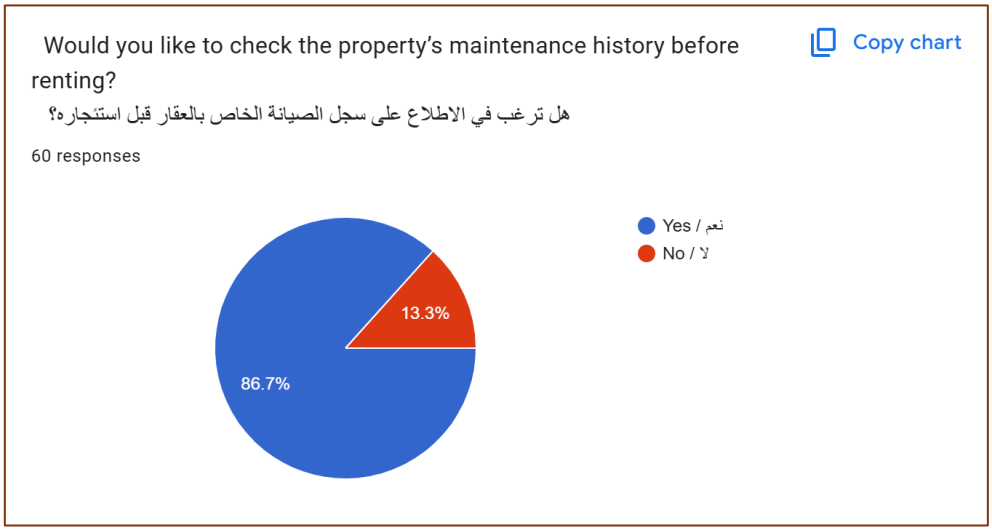

Figure 16. Chart showing tenant interest in viewing a property's maintenance history before renting.

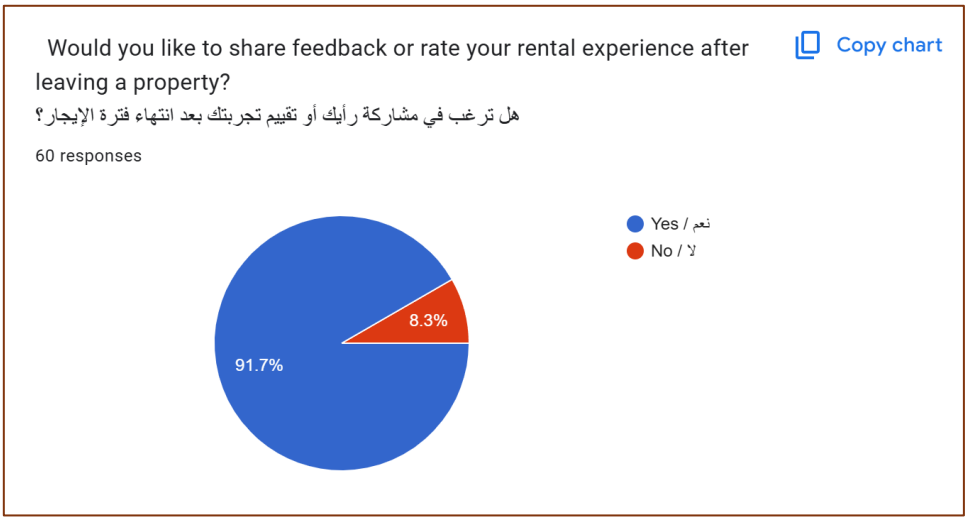

Figure 17. Chart shows tenant willingness to share feedback or rate their rental experience after leaving a property.

Landlords Answers

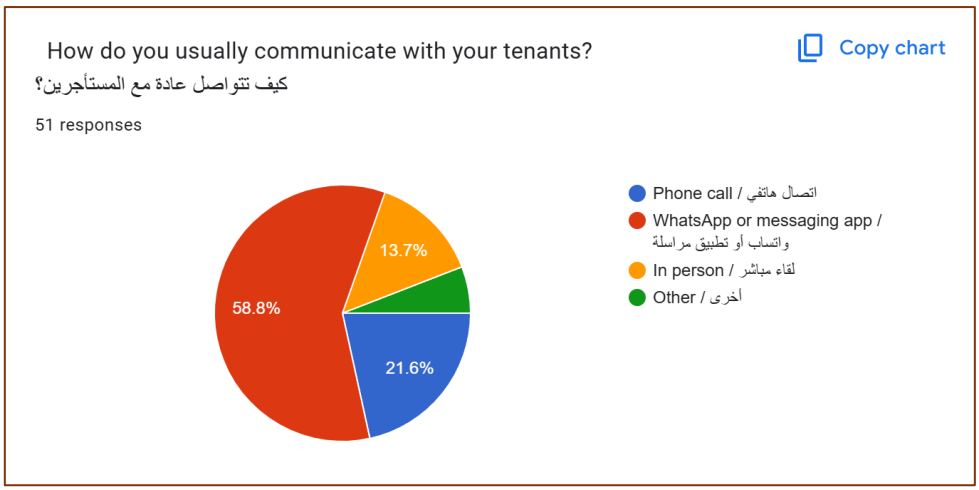

Figure 18. Chart showing landlords' preferred methods of communication with tenants.

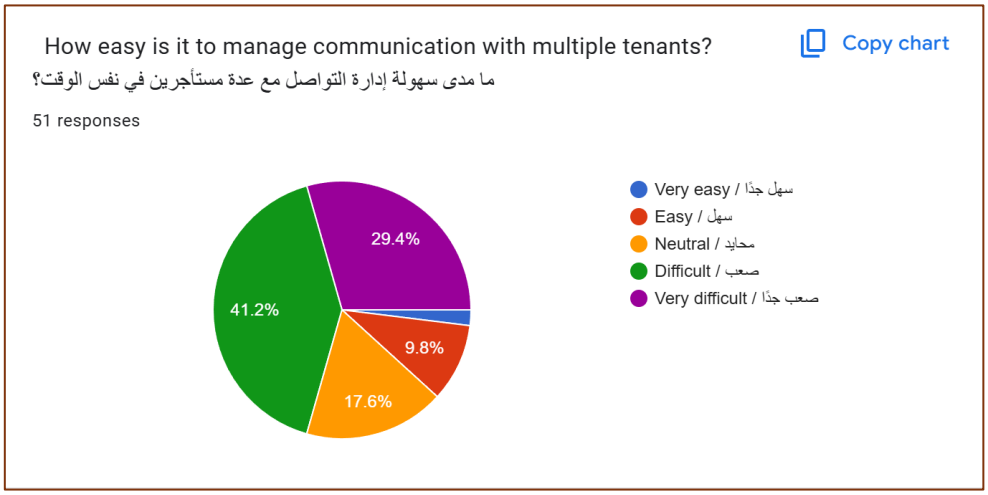

Figure 19. Chart showing landlords' responses on the ease of managing communication with multiple tenants.

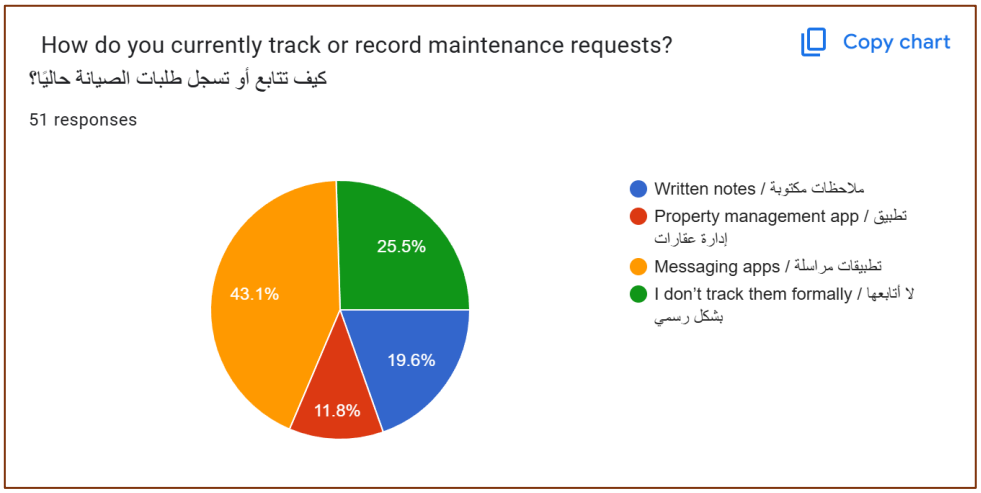

Figure 20. Chart showing landlords' methods for tracking or recording maintenance requests.

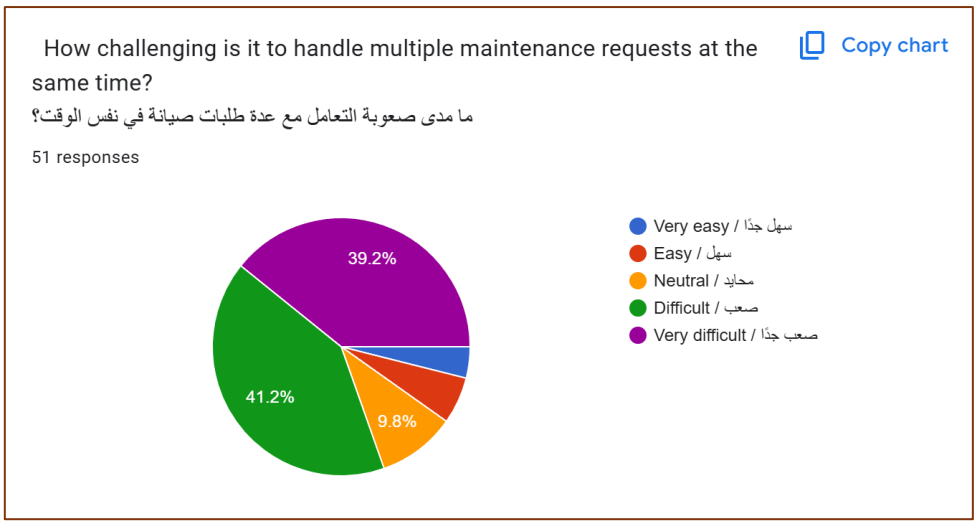

Figure 21. Chart showing landlords' responses on the difficulty of handling multiple maintenance requests simultaneously.

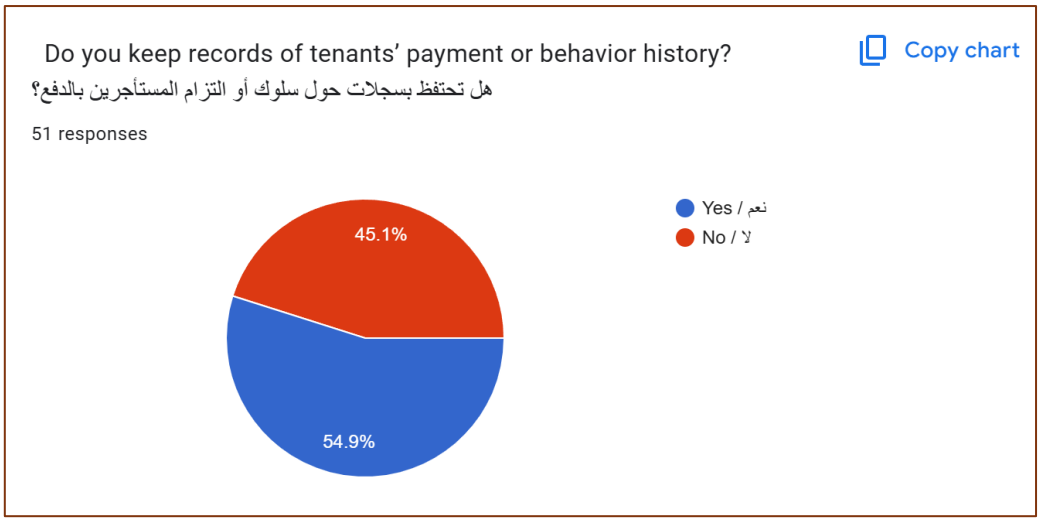

Figure 22. Chart showing landlords' responses on keeping records of tenants' payment or behavior history.

What are the most time-consuming tasks in managing your rental properties?
ما أكثر المهام التي تستهلك وقتك في إدارة عقاراتك؟

7 responses

تنظيم سجلات واحصائيات المستأجرين بشكل دوري

ادارة التاجير بين العقارات

ترتيب مواعيد الصيانة

مراجعة الامور المالية و معرفة اوقات الصيانة المناسبة للسكان

ارسال تنبيهات للمستأجرين

اعداد قوائم الايجار

مراجعة قوائم المستأجرين ومواعيد الإيجارات

Figure 23. landlords' written responses on the most time-consuming tasks in managing rental properties.